535106 LLM Team Project: Generalizing Mechanistic Hallucination Detection: Applying ReDeEP to the HalluRAG Benchmark

張周芳　賴子永　林致葆　歐陽元貞
Institute of Electrical and Computer Engineering
National Yang Ming Chiao Tung University
{fangfangirl037.ee14, ee314511062.ee14, elimath9104.ee14, oyyj.ee14}@nycu.edu.tw

1 Project Overview

1.1 Profile

Track: 3. Application

Abstract and Project Goal:

研究主要在重現並驗證 ReDeEP 論文中提出的 RAG（Retrieval-Augmented Generation）模型幻覺檢測方法。該方法透過「機制可解釋性」（Mechanistic Interpretability）分析模型內部的注意力（Attention Heads）與前饋層（Feed-Forward Networks, FFNs）運作機制，提出了兩個關鍵指標：外部內容分數（External Context Score, ECS）與參數化知識分數（Parametric Knowledge Score, PKS），以判定生成結果是否有根據眞實資料生成。

論文在 RAGTruth 與 Dolly 等資料集上展現了優異的幻覺檢測效果，顯示 ReDeEP 能有效區分「引用外部眞實知識的生成」與「來自內部錯誤推斷的幻覺」。然而，該方法的穩健性與泛化能力尚未在情境更複雜的資料集上得到驗證。

HalluRAG 是近期提出的 RAG 幻覺檢測基準，專注於模型在面對**知識邊界之外（knowledge cut-off）**資訊時的表現，這類幻覺通常發生於模型試圖回答訓練資料中未涵蓋或發生於模型完成訓練後的新事件時。即使 RAG 能檢索補充資料，模型仍可能錯誤地依賴自身舊有知識而產生「模型不知道自己不知道（unknown unknowns）」的自信錯誤。因此本專題將 ReDeEP 方法應用於 HalluRAG 資料集進行延伸。

TL;DR: 重現 ReDeEP 論文的 Hallucination Detection，將資料集變成 HalluRAG，以測試其在其他資料集偵測幻覺時的泛化性與穩健性。

1.2 Motivation

Why is the problem interesting?

大型語言模型（Large Language Models, LLMs）在各領域的應用中展現了卓越的語言生成與知識整合能力，然而「幻覺」（hallucination）問題仍是限制其可靠性與落地應用的關鍵挑戰之一。當模型內部的參數化知識（parametric knowledge）與外部檢索的上下文內容（retrieved context）發生衝突時，即使檢索結果正確，模型仍可能生成錯誤或虛構的內容，進而產出錯誤資料，此風險在醫療、金融、法律與教育等高度依賴眞實資訊的領域中，影響更爲嚴重。

雖然 RAG（Retrieval-Augmented Generation）框架的提出原意是爲了緩解此問題，使模型能根據外部知識生成更可靠的答案，但實際上模型仍常錯誤地忽略外部檢索內容，轉而依

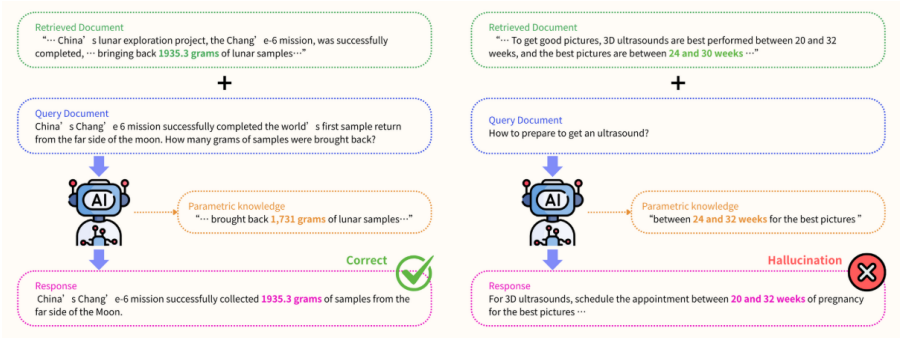

Figure 1: Two examples of RAG where the retrieved document is correct but conflicts with para metric knowledge. The left example shows a correct response based on external knowledge, while the right example demonstrates hallucination despite accurate external context.

賴其舊有的參數化知識進行推斷。這顯示出模型在判斷自身資料可靠性的能力仍然有進步空間。

如圖 1 所示，即爲一個模型未能有效參考 RAG 所提供之外部文件內容的案例。儘管檢索結果本身是正確的，模型卻仍因過度依賴內部知識而產生錯誤回應，進而導致幻覺的發生。

因此，嘗試建構一套幻覺偵測器對使用者提供反饋爲一重要工具。此項技術藉由幻覺偵測機制，協助使用者判斷模型輸出的可信任度（trustworthiness），亦有助於提升 LLM 生成資料在應用於眞實場景中的價值與可靠性。

Critical Challenges

Challenge 1: 機制可解釋性方法的泛化（Generalization of Mechanistic Interpretability）

ReDeEP 依賴對模型內部注意力頭（Copying Heads）與知識前饋網路（Knowledge FFNs）的觀察，分析模型如何整合外部與內部知識。然而，這些機制特徵是否能泛化至新的資料集（例如 HalluRAG）的幻覺仍未被驗證。若模型在新知識情境下產生偏移，則原有的可解釋性指標可能失效。

Challenge 2: 可重現性與穩健性（Reproducibility and Robustness）

重現 ReDeEP 涉及高度複雜的模型分析程序，包括多層 Transformer 結構的注意力追蹤與特徵迴歸計算。任何細微的實作差異（如超參數設定或模型版本）都可能影響最終結果。此外，當方法應用於更大規模、跨領域的資料集時，如何確保計算效率與指標穩定性也是重要挑戰。

Why the problem remains open?

目前多數幻覺檢測方法仍偏重於「輸出層級」的語義一致性檢查，如 SelfCheckGPT Manakul et al. [2023] 或 RefChecker Hu et al. [2024]，而較少探討模型內部機制與幻覺行爲之間的關聯。ReDeEP Sun et al. [2024] 首次嘗試透過機制可解釋性方法揭示 RAG 幻覺的成因，並提出基於 ECS/PKS 的定量檢測架構。然而，該方法僅在靜態資料集上測試，其在「知識邊界之外」場景中的可行性仍屬未知。

Relevant Prior Works and SOTA

近兩年與本專題最相關的論文包括:

- **Sun et al. [2024]** —提出 *ReDeEP*，以迴歸模型內部外部知識分數的方式進行幻覺檢測;
- **Ridder and Schilling [2024]** —發表 *HalluRAG*，建立針對知識邊界之外幻覺的標準化評估資料集;
- **Lewis et al. [2020]** —提出 *RAG* 框架，開啓了檢索增強生成的研究方向。

根據前述文獻，在現有研究當中我們認爲 **ReDeEP** 仍是目前在「基於模型內部機制」進行幻覺檢測的最先進方法，而本期末專題將進一步檢驗其在 HalluRAG 資料集上的泛化能力與應用潛力。

2 Problem Formulation

本專題的任務設定爲「檢索增強生成」（Retrieval-Augmented Generation, RAG）場景下的幻覺檢測（hallucination detection），目標是復現並驗證一個能夠準確評估模型生成內容眞實性，並預測其可能出現幻覺機率的檢測器（detector）。

整體任務爲一個多元線性迴歸（multiple linear regression）問題。根據 ReDeEP 論文的假設，模型生成的每一段回覆可由兩種特徵共同決定：

1. 來自模型內部參數化知識（parametric knowledge）的貢獻，記爲 PKS；
2. 來自外部檢索內容（retrieved context）的貢獻，記爲 ECS。

我們的目標是透過迴歸學習找到一組最佳的迴歸係數 α 與 β，使得下列線性方程式能最準確地預測眞實的幻覺標籤（ground-truth hallucination label）：

$$\text{Hallucination Score} = \alpha \cdot P - \beta \cdot E$$

其中，P 代表由模型內部前饋神經網路（Feed-Forward Networks, FFNs）計算出的「參數化知識分數」（Parametric Knowledge Score, PKS），而 E 則代表由注意力機制中複製頭（Copying Heads）所量化的「外部內容分數」（External Context Score, ECS）。

在每一次生成過程中，大型語言模型（LLM）的輸入包括使用者查詢（query）以及從外部知識庫檢索到的文件（retrieved documents），形成標準的 RAG 輸入架構。模型根據這些資料生成回應，我們則從其內部層結構中提取 FFN 與注意力層的相關特徵，分別計算 P 與 E。

監督訊號（supervision signal）來自於 HalluRAG 資料集所提供的人工標註二元標籤，明確標示模型生成的每一段回應是否是幻覺（1 表幻覺、0 表眞實）。透過最小化預測分數與標註結果的誤差，學得最佳的 α 與 β，可以建立一個可量化且具可解釋性的幻覺偵測模型。

3 Empirical Evaluation

3.1 Performance Metrics

我們專注於 ReDeEP 方法在 **幻覺偵測（Hallucination Detection）** 任務上的效能評估，我們將從兩個方向評估模型的表現：偵測準確度與內部機制利用。

(1) 偵測準確度（Detection Accuracy Metrics）

- **評估指標:** AUC、PCC、$F1$ Score、Accuracy。
- **評估目的:** 衡量 ReDeEP 模組在二元分類任務（幻覺 vs. 非幻覺）中，是否能正確區分「眞實（Truthful）」生成與「幻覺（Hallucinated）」生成。
- **設計考量:** 由於資料集中幻覺樣本比例可能不均衡，因此選擇 $F1$ 分數作爲主要評估指標，以平衡 Precision 與 Recall 的貢獻，提供對整體檢測性能的更準確衡量。

(2) 內部機制利用指標（Internal Mechanistic Utilization Metrics）

- **評估指標:**
  - **外部上下文分數（External Context Score, ECS）:** 衡量 LLM 內部 **注意力頭（Attention Heads）** 對檢索外部上下文資訊的依賴程度。
  - **參數化知識分數（Parametric Knowledge Score, PKS）:** 衡量 LLM **前饋層（Feed-Forward Networks, FFNs）** 對內部參數化知識的使用程度。
- **設計考量:** ECS 與 PKS 爲 ReDeEP 方法的理論核心，分別對應模型對外部檢索知識與內部參數化知識的依賴關係。透過觀察這兩個分數的變化，可檢驗模型在不同資料集（如 HalluRAG）下是否維持相似的知識整合模式，進而驗證 ReDeEP 在可解釋性（Mechanistic Interpretability）與泛化能力（Generalization）上的穩健性。

3.2 Baseline Methods

本專題並非著重於設計新的幻覺偵測模型，而是聚焦於 **資料層面的延伸與再評估**。我們的主要目標是將 ReDeEP 方法套用至新的基準資料集 **HalluRAG**，以分析其在不同資料特性下的表現變化與泛化能力。

在原始論文中，ReDeEP 於 **RAGTruth** 與 **Dolly** 資料集上進行實驗，這兩者皆屬於靜態且知識範圍明確的資料集。相比之下，**HalluRAG** 專注於「知識邊界之外（knowledge cut-off）」的情境，即模型面對訓練後才出現的新知識時的生成行爲。

因此，本專題的 baseline 設定如下：

- **Baseline: 原始 ReDeEP + RAGTruth**（對照組）—代表模型在靜態資料集上的表現。
- **Proposed: ReDeEP + HalluRAG**（實驗組）—檢驗相同方法於動態知識場景下的穩健性與泛化能力。

專題將沿用 ReDeEP 的模型架構與計算流程（ECS/PKS 與線性迴歸偵測器），僅更換資料集以觀察方法在不同知識分佈下的適應性（dataset shift robustness）。

3.3 Benchmark Datasets

爲同時實現 ReDeEP 的重現驗證與泛化能力分析，本研究採用以下兩個資料集：

- **RAGTruth** (Niu et al. [2024])
  - 用途：作爲基準資料集，用於重現與驗證 ReDeEP 原論文所提出之幻覺檢測效能與準確度。
  - 資料集描述： RAGTruth 是一個高品質、經人工標註的 RAG 幻覺檢測基準資料集，每筆資料包含查詢（query）、檢索內容（retrieved context）、模型生成回應（generation），以及對應的二元幻覺標籤（0：事實正確，1：幻覺）。該資料集主要用於評估模型在「給定檢索內容」情境下的事實一致性。
- **HalluRAG** (Ridder and Schilling [2024])
  - 用途：用於評估 ReDeEP 方法在極端情境下的穩健性（robustness）與跨場景泛化能力（generalization）。
  - 資料集描述： HalluRAG 專注於「知識截止點（knowledge cut-off）」情境，模擬模型被詢問其訓練資料時間範圍之外的新事件時的行爲。在此設定下，模型必須完全依賴檢索結果作答，用以檢驗模型是否會自信地生成實際不存在的內容，更貼近眞實世界中「模型不知道自己不知道（unknown unknowns）」所導致的幻覺問題。
- **兩個資料集的差異**
  - HalluRAG 中的資料保證不是語言模型曾經的訓練資料
  - 對於模型內部知識不包含、且外部檢索亦未提供相關資訊的問題，HalluRAG 允許模型輸出「我不知道」或等價的不確定回應，並將其標記爲非幻覺；相較之下，RAGTruth 則一律要求模型產生具體回答，因而導致幻覺輸出。

4 Methodology

4.1 階段一：資料處理（Data Processing）

本研究同時使用 RAGTruth 與 HalluRAG 兩個資料集。由於 RAGTruth 的欄位結構已可直接滿足本研究之模型輸入需求，而 HalluRAG 的資料格式與標註方式不同，無法直接與 RAGTruth 共同進行後續分析。因此，我們在資料處理階段包含兩個子步驟：（1）HalluRAG 資料集正規化，使其欄位與語義對齊 RAGTruth；（2）對兩個資料集施以統一的 chunk-level 前處理，使後續特徵計算能以相同 level 比較 prompt 與 response 的資訊關係。

4.1.1 HalluRAG 資料集正規化（Normalization）

在此步驟中，我們直接使用 HalluRAG 原始資料所提供的 prompt 與模型生成回應 response，不對任何文字內容進行重新生成或改寫。對於 prompt，我們僅保留 role 爲 `user` 的輸入內容，並將多段使用者輸入依序串接，形成單一的使用者提示文字；對於 response，則完整保留模型原始輸出作爲分析對象。

此外，HalluRAG 提供 sentence-level 的幻覺標註資訊。爲了與後續的 chunk-level 分析相容，我們將其整理爲 response-level 的幻覺標籤，並進一步利用累積句子（cumulative sentence）之差分方式，回推出第一個 hallucinated 句子所對應的字元層級幻覺區段（character-level hallucination span）。此 span 之定位結果會透過精確比對或模糊比對進行驗證，以降低因空白或標點差異造成的對齊誤差。完整流程如 Algorithm 1 所示。

Algorithm 1: HalluRAG Minimal Dataset Construction

Input: Raw HalluRAG dataset $\mathcal{D}$
Output: Minimal dataset $\mathcal{D}_{min}$
Initialize empty dataset $\mathcal{D}_{min}$
foreach *entry* $e \in \mathcal{D}$ **do**
  Prompt extraction:
  Retrieve the original prompt structure P_e
  Extract and concatenate all prompt segments with role = `user` to obtain p_e
  if p_e *is empty* **then**
    continue
  Response extraction:
  Retrieve the original model-generated response R_e
  Set $r_e \leftarrow R_e$
  Hallucination label assignment:
  Assign response-level label

$$y_e = \begin{cases} 1, & \exists s_i \in S_e \text{ such that label}(s_i) = \texttt{hallucinated} \\ 0, & \text{otherwise} \end{cases}$$

  Hallucination span recovery:
  Initialize hallucination span $s_e \leftarrow \varnothing$
  Initialize previous cumulative sentence $p \leftarrow \emptyset$
  foreach *sentence* $s_i \in S_e$ *(in order)* **do**
    if s_i *is labeled hallucinated* **then**
      Compute delta text $\Delta_e = s_i^{cum} \setminus p$
      Locate Δ_e in response r_e to obtain character span (l_e, r_e)
      Verify span alignment using exact or fuzzy matching
      if *verification succeeds* **then**
        Set $s_e \leftarrow (l_e, r_e)$
      break
    Update $p \leftarrow s_i^{cum}$
  Append (p_e, r_e, y_e, s_e) to $\mathcal{D}_{min}$
return $\mathcal{D}_{min}$

4.1.2 統一的 Chunk-level 前處理（Unified Chunk-level Preprocessing）

完成 HalluRAG 正規化後，兩個資料集在欄位語義上已可對齊。接著我們對 RAGTruth 與 HalluRAG 實作完全一致的 chunk-level 前處理，並由此開始建立實際的特徵提取的輸入。我們的目標是檢測 response 中的內容是否受到 prompt 的支持，因此需要將 prompt 與 response 切分爲較細緻且可對齊的語義單位，以利後續逐 chunk 比對外部資訊與生成內容的關係。

我們採用 sentence-aware 的切分策略：先依自然語言分句，例如：句號、逗號、分號.....，再將相鄰句子合併爲不超過最大長度的 chunk；同時引入 chunk overlap 保留上下文連續性；最後調整 chunk 邊界避免落在英文單字中間，以提升 embedding 與相似度計算的穩定性。切分流程如 Algorithm 2 所示。

Algorithm 2: Sentence-aware Chunk-level Preprocessing

Input: Text sequence T, maximum chunk length L, overlap size O
Output: A list of character-level chunk spans $\mathcal{C} = \{(s_i, e_i)\}$
Split T into an ordered list of sentences using a sentence tokenizer
Compute character-level spans for each sentence in T
Initialize empty chunk list $\mathcal{C}$
Initialize current chunk start $c_s \leftarrow$ `null`, current chunk end $c_e \leftarrow$ `null`, current length $\ell \leftarrow 0$
foreach *sentence span* (s, e) *in order* **do**
  if c_s *is* `null` **then**
    Set $c_s \leftarrow s$, $c_e \leftarrow e$, $\ell \leftarrow e - s$
    continue
  if $\ell + (e - s) \leq L$ **then**
    Extend current chunk: $c_e \leftarrow e$, $\ell \leftarrow c_e - c_s$
  else
    Adjust chunk boundary to nearest safe character to avoid splitting inside a word
    Append (c_s, c_e) to $\mathcal{C}$
    Compute next chunk start $n_s \leftarrow \max(c_e - O, 0)$
    Adjust n_s forward to the nearest safe character boundary
    Set $c_s \leftarrow n_s$, $c_e \leftarrow e$, $\ell \leftarrow c_e - c_s$
if c_s *is not* `null` **then**
  Append (c_s, c_e) to $\mathcal{C}$
return $\mathcal{C}$

4.2 階段二：特徵提取（Feature Extraction）

在完成統一的 chunk-level 資料處理後，我們進一步使用這些資料從模型內部狀態提取用於幻覺檢測的特徵，包含三部分：（1）Copy Head 找尋，用以識別注意力機制 (attention map) 中具有「複製行爲」傾向的 head；（2）外部上下文分數（ECS），衡量模型生成內容與檢索/輸入上下文在語義層面的對齊程度；（3）參數化知識依賴程度（PKS），反映模型在生成過程中依賴內部參數知識的強度。這些特徵將在後續模型訓練階段被用作輸入變數，以進行幻覺預測。

4.2.1 Copy Head 選取與評估

在計算外部上下文（ECS）之前，我們首先需要從模型的注意力機制中挑選出最具代表性的 Attention Heads。由於大型語言模型通常由 L 個 Transformer 層堆疊而成，每層包含 H 個 Attention Heads，對所有 Attention Heads 進行計算將導致極高的計算成本，且並非所有 Attention Heads 都具有與幻覺檢測相關的行爲特徵。

因此，我們僅選擇所謂的 Copy Heads，即注意力分布傾向將 prompt 中的 token 對齊或複製至 response 的位置，作爲後續特徵計算的基礎。爲了量化此行爲，我們針對每一層、每一個 Attention Head，統計其在所有生成位置上的注意力行爲，並設計四個指標進行評估：

- **前一 token 複製比例（Prev-Token Copy Ratio）：** 衡量注意力是否偏向前一位置，反映 shifted copy 行爲。
- **自身 token 複製比例（Self-Token Copy Ratio）：** 衡量注意力是否集中於當前位置。
- **注意力峰值比例（Peak Attention Ratio）：** 反映注意力是否高度集中於單一來源 token。
- **注意力熵（Attention Entropy）：** 衡量注意力分布的分散程度，熵值越低代表行爲越接近複製。

上述指標經過全模型範圍的正規化後，我們以加權方式將其整合爲單一的 Copy Score，並依分數由高至低排序，選出前 K 個最具代表性的 Copy Heads，其中，我們設定 $K = L$，使 Copy Head 特徵的數量與後續所計算之 PKS（Parametric Knowledge Score）在數量上

保持一致。此設計可確保不同特徵在模型訓練時具有相同的維度，避免因特徵數量差異過大而造成權重學習上的偏差。

這些 Copy Heads 將被用於後續的 ECS 特徵計算。完整流程如 Algorithm 3 所示。

Algorithm 3: Attention-based Copy Head Identification

Input: Transformer model M with L layers and H attention heads per layer;
A set of input texts $\mathcal{T}$;
Number of selected heads K
Output: Top-K Copy Heads $\mathcal{H}_{copy}$
Initialize empty statistics table for all head pairs (ℓ, h)
foreach *layer index* $\ell = 1$ *to* L **do**
  Register a forward hook on the self-attention module of layer ℓ
  Initialize counters for each head h: self-copy count, prev-copy count, other-copy count, entropy sum, peak sum
  foreach *text* $t \in \mathcal{T}$ **do**
    Perform forward pass with attention outputs enabled
    Retrieve attention weights $W \in \mathbb{R}^{B \times H \times T \times S}$
    foreach *batch* b*, head* h*, target position* i **do**
      Let $j = \arg\max W_{b,h,i,:}$ be the source position
      Accumulate entropy and peak attention statistics
      if $j = i$ **then**
        Increment self-copy counter
      else if $j = i - 1$ **then**
        Increment prev-copy counter
      else
        Increment other-copy counter
  Remove the forward hook
  Compute ratio-based metrics for each head in layer ℓ
Normalize attention entropy and peak ratios across all heads
foreach *head* (ℓ, h) **do**
  Compute Copy Score:

$$\text{Copy score} = \alpha \cdot \text{PrevCopy} + \beta \cdot \text{SelfCopy} + \gamma \cdot \text{Peak} + \delta \cdot (1 - \text{Entropy})$$

Sort all heads by Copy Score in descending order
Select top-K heads as $\mathcal{H}_{copy}$
return $\mathcal{H}_{copy}$

4.2.2 外部上下文（ECS）計算

在選定 Copy Heads 後，我們計算外部上下文分數（External Context Score, ECS），用以衡量模型在生成每個 response chunk 時，是否眞正參考了 prompt 中的外部內容。整體流程分爲三步。

第一步，我們將 prompt 與 response 依照模型的 chat template 進行拼接，並重新送入模型以取得所選 Copy Heads 的 attention maps。透過這些 attention maps，我們可以觀察每個 response chunk 在生成時主要關注哪些 prompt chunk。

第二步，對於每個 response chunk，我們在 attention map 中計算其對所有 prompt chunks 的注意力總和，並選取注意力分數最高的 prompt chunk 作爲「最可能的引用來源」。此步驟將 token-level 的注意力訊號轉換爲 chunk-level 的對齊關係，從而降低計算複雜度並提升可解釋性。

第三步，由於「高注意力」不代表「實際引用」，因此我們使用語意向量模型（BGE embedding）計算該 response chunk 與其對齊的 prompt chunk 之語意相似度，作爲該 Copy Head 下的 ECS 分數。也就是說，只有在注意力集中且語意相近時，才視爲模型有採用外部資訊。完整流程如 Algorithm 4 所示。

Algorithm 4: Chunk-level External Context Score (ECS) Computation

Input: Tokenized full input sequence $x = [x_1, \ldots, x_n]$ formed by concatenating prompt and response;
Prompt token span set $\mathcal{P} = \{[p_s, p_e)\}$;
Response token span set $\mathcal{R} = \{[r_s, r_e)\}$;
Prefix length m (prompt-only token length);
Selected Copy Heads $\mathcal{H}_{copy} = \{(\ell, h)\}$;
Embedding model $\mathrm{emb}(\cdot)$
Output: Chunk-level ECS scores $\mathrm{ECS}_{\ell,h}(r)$
Run the LLM on x with attention outputs enabled to obtain attention tensors $\mathbf{A}^{(\ell)} \in \mathbb{R}^{1\times H\times n\times n}$
foreach *response chunk span* $r = [r_s, r_e) \in \mathcal{R}$ **do**
  Decode response chunk:

$$t_r \leftarrow \mathrm{decode}(x[r_s : r_e])$$

  foreach *Copy Head* $(\ell, h) \in \mathcal{H}_{copy}$ **do**
    Extract chunk-to-prompt attention map:

$$\mathbf{W} \leftarrow \mathbf{A}^{(\ell)}[1, h, r_s : r_e, 1 : m]$$

    Initialize best prompt span $p^\star \leftarrow \varnothing$
    foreach *prompt chunk span* $p = [p_s, p_e) \in \mathcal{P}$ **do**
      if $p_e > m$ **then**
        continue
      Compute attention mass:

$$s(p) \leftarrow \sum \mathbf{W}[:, p_s : p_e]$$

    Select the most-attended prompt chunk:

$$p^\star \leftarrow \arg\max_{p\in\mathcal{P}} s(p)$$

    Decode selected prompt chunk:

$$t_{p^\star} \leftarrow \mathrm{decode}(x[p^\star_s : p^\star_e])$$

    Compute semantic similarity (cosine) as ECS:

$$\mathrm{ECS}_{\ell,h}(r) \leftarrow \cos\big(\mathrm{emb}(t_r), \mathrm{emb}(t_{p^\star})\big)$$

return $\{\mathrm{ECS}_{\ell,h}(r)\}$

4.2.3 參數化知識分數（PKS）計算

除了 ECS 之外，我們進一步計算參數化知識分數（Parametric Knowledge Score, PKS），用以衡量模型在生成每個 response chunk 時，於不同 Layer 中對「內部參數化知識」的依賴程度，整體流程同樣分爲三個步驟。

第一步，我們將 prompt 與 response 依照模型的 chat template 進行拼接，並重新送入模型以取得各層的 hidden states。透過這些 hidden states，我們可以觀察模型在不同層中對同一 response chunk 的影響。

第二步，對於每個 response chunk，我們使用 Logit Lens 將相鄰兩層（Layer i 與 Layer i+1）的 hidden states 投影回詞彙空間，得到對應的 logits 分布。

第三步，計算這兩個 logits 分布之間的 Jensen–Shannon Divergence（JSD），用來量化相鄰層在該 chunk 上的輸出差異。由於 JSD 能夠衡量兩個機率分布的相似程度，因此較小的 JSD 代表 Layer i 與 Layer i+1 的輸出分布高度相近，表示模型在該層並未引入顯著的新資訊，而是傾向沿用既有的內部表示；反之，較大的 JSD 則反映模型在層間進行了較明顯的特徵更新，參考了模型內部資料。JSD 的計算結果就是我們對每一 response chunk 在每一層取得的 PKS 分數。我們對所有 Transformer 中的 FFNs 都計算 PKS。完整流程如 Algorithm 5 所示。

Algorithm 5: Layer-wise Parametric Knowledge Score (PKS) via Logit Lens and JSD

Input: Tokenized full input sequence $x = [x_1, \ldots, x_n]$;
Response chunk spans $\mathcal{R} = \{[r_s, r_e)\}$;
Selected layer set $\mathcal{L} = \{0, \ldots, L-1\}$ (or a subset);
Final normalization $\mathrm{Norm}(\cdot)$ and language modeling head $\mathrm{LMHead}(\cdot)$
Output: PKS scores $\mathrm{PKS}_i(r)$ for each layer $i \in \mathcal{L}$ and response chunk $r \in \mathcal{R}$
Run the LLM on x with hidden states to obtain $\mathbf{H}^{(i)} \in \mathbb{R}^{1\times n\times d}$ for each layer i
foreach *response chunk span* $r = [r_s, r_e) \in \mathcal{R}$ **do**
  foreach *layer* $i \in \mathcal{L}$ **do**
    if $i+1$ *exceeds available hidden states* **then**
      break
    Extract chunk hidden states:

$$\mathbf{h}^{(i)} \leftarrow \mathbf{H}^{(i)}[1, r_s : r_e, :], \quad \mathbf{h}^{(i+1)} \leftarrow \mathbf{H}^{(i+1)}[1, r_s : r_e, :]$$

    Logit Lens projection:

$$\mathbf{z}^{(i)} \leftarrow \mathrm{LMHead}(\mathrm{Norm}(\mathbf{h}^{(i)})), \quad \mathbf{z}^{(i+1)} \leftarrow \mathrm{LMHead}(\mathrm{Norm}(\mathbf{h}^{(i+1)}))$$

    Convert logits to distributions:

$$\mathbf{p} \leftarrow \mathrm{softmax}(\mathbf{z}^{(i)}), \quad \mathbf{q} \leftarrow \mathrm{softmax}(\mathbf{z}^{(i+1)})$$

    Compute Jensen–Shannon Divergence:

$$\mathbf{m} \leftarrow \tfrac{1}{2}(\mathbf{p}+\mathbf{q}), \quad \mathrm{JSD}(\mathbf{p},\mathbf{q}) \leftarrow \tfrac{1}{2}\mathrm{KL}(\mathbf{p}\|\mathbf{m}) + \tfrac{1}{2}\mathrm{KL}(\mathbf{q}\|\mathbf{m})$$

    Aggregate JSD over tokens in chunk:

$$\mathrm{PKS}_i(r) \leftarrow \mathrm{mean}\big(\mathrm{JSD}(\mathbf{p},\mathbf{q})\big)$$

return $\{\mathrm{PKS}_i(r)\}$

4.3 階段三：模型訓練與測試（Model Training and Evaluation）

在完成階段二的特徵提取後，我們使用外部上下文（ECS）與參數化知識分數（PKS）作爲輸入特徵，訓練幻覺檢測模型，並於測試集上進行評估。爲兼顧效能、穩健性與可解釋性，我們比較兩種線性方法：(1) 邏輯迴歸（Logistic Regression）；(2) 非負最小平方法（Non-Negative Least Squares, NNLS）。兩種方法皆在相同的資料切分、特徵選擇、決策閾值學習與評估流程下進行實驗，以確保比較的公平性。

所有模型皆於 *chunk-level* 進行訓練，並同時於 *chunk-level* 與 *response-level* 進行評估。

4.3.1 特徵建構與選擇

對於每一個 response chunk，我們保留由階段二所計算的所有 ECS heads 與 PKS layers 作爲候選特徵。爲降低模型複雜度並提升穩健性，我們首先於訓練集上，分別計算每一個 ECS head 與每一個 PKS layer (即爲所有 Transformer Layer) 單獨對幻覺標籤的 AUC，並依 AUC 由高至低進行排序。

接著，我們透過 grid search，評估不同 Top-N ECS heads 與 Top-K PKS layers 的組合。對於每一組 (N, K)，我們皆重新訓練對應的線性模型，並於訓練集上完成特徵正規化與決策閾值學習，再於測試集上進行評估。模型選擇以 response-level AUC 作爲主要依據。

4.3.2 邏輯迴歸（Logistic Regression）

在 Logistic Regression 設定中，我們將特徵選擇與模型訓練納入同一個模型選擇流程中進行。由於後續模型將多個 ECS heads 與 PKS layers 分別加總爲單一特徵（$\mathrm{ECS}_{\mathrm{sum}}$ 與 $\mathrm{PKS}_{\mathrm{sum}}$），若直接使用所有特徵進行彙整，可能引入冗餘或判別力不足的訊號，進而影響線性模型的穩定性與泛化能力。因此，在進行特徵彙整之前，我們先進一步進行特徵子集選擇，以降低模型複雜度並提升穩健性。

因此，我們首先僅於訓練集上，分別計算每一個 ECS head 與每一個 PKS layer 單獨對幻覺標籤的 AUC，並依 AUC 由高至低進行排序，作爲後續特徵子集選擇的依據。

接著，我們透過 grid search 評估不同 Top-N ECS heads 與 Top-K PKS layers 的組合。對於每一組 (N, K)，我們重新建構對應的特徵集合，將選定的 ECS 與 PKS 特徵分別加總爲 $\mathrm{ECS}_{\mathrm{sum}}$ 與 $\mathrm{PKS}_{\mathrm{sum}}$，作爲 Logistic Regression 的輸入特徵，分別表示模型對外部上下文與內部參數化知識的整體依賴程度。

在模型訓練前，我們僅使用訓練集估計 Min–Max scaling 參數，並將相同的正規化轉換套用至測試集，以消除特徵尺度差異對係數學習造成的偏頗，同時避免測試集資訊洩漏。

對於每一組 (N, K)，我們於訓練集上訓練對應的 Logistic Regression 模型，並輸出每一個 response chunk 對應的連續幻覺分數（decision function）。對於每一個 response chunk r_j，其幻覺分數定義爲：

$$\mathrm{Score}(r_j) = \alpha \cdot \mathrm{PKS}_{\mathrm{sum}}(r_j) - \beta \cdot \mathrm{ECS}_{\mathrm{sum}}(r_j) \tag{1}$$

其中 α 與 β 分別爲 $\mathrm{PKS}_{\mathrm{sum}}$ 與 $\mathrm{ECS}_{\mathrm{sum}}$ 所對應的權重，皆由 Logistic Regression 於訓練集上學習得到。

爲將連續分數轉換爲二元預測，決策閾值 (threshold) 同樣僅於訓練集上，利用 ROC 曲線中的 *Youden's J statistic* 進行學習，即最大化

$$J = \mathrm{TPR} - \mathrm{FPR}.$$

所有模型皆於測試集上進行評估，並以 response-level AUC 作爲模型選擇的主要依據。最終，我們選定表現最佳的 (N^*, K^*) 作爲最終模型設定。

在測試階段中，該模型所學得的參數與決策閾值皆固定不變，僅用於模型評估。

完整流程如 Algorithm 6 所示。

Algorithm 6: Logistic Regression-based Hallucination Scoring with Feature Selection

Input: Training set $\mathcal{D}_{\mathrm{train}}$, Test set $\mathcal{D}_{\mathrm{test}}$;
Candidate ECS head set $\mathcal{H}_{\mathrm{cand}}$;
Candidate PKS layer set $\mathcal{L}_{\mathrm{cand}}$
Output: Hallucination scores and predicted labels at chunk-level and response-level
Feature ranking (train only):
foreach $(\ell, h) \in \mathcal{H}_{\mathrm{cand}}$ **do**
  Compute AUC of $\mathrm{ECS}_{\ell,h}$ on $\mathcal{D}_{\mathrm{train}}$
foreach $i \in \mathcal{L}_{\mathrm{cand}}$ **do**
  Compute AUC of PKS_i on $\mathcal{D}_{\mathrm{train}}$
Sort ECS heads and PKS layers by descending AUC to obtain $\mathcal{H}_{\mathrm{rank}}$ and $\mathcal{L}_{\mathrm{rank}}$

Grid search over feature subsets:
foreach (N, K) **do**
  Select Top-N ECS heads $\mathcal{H} \subset \mathcal{H}_{\mathrm{rank}}$
  Select Top-K PKS layers $\mathcal{L} \subset \mathcal{L}_{\mathrm{rank}}$
  foreach $r_j \in \mathcal{D}_{\mathrm{train}}$ **do**
    Compute $\mathrm{PKS}_{\mathrm{sum}}(r_j) = \sum_{i \in \mathcal{L}} \mathrm{PKS}_i(r_j)$
    Compute $\mathrm{ECS}_{\mathrm{sum}}(r_j) = \sum_{(\ell,h) \in \mathcal{H}} \mathrm{ECS}_{\ell,h}$
  Apply Min–Max normalization using $\mathcal{D}_{\mathrm{train}}$
  Train Logistic Regression to learn (α, β)
  Compute continuous hallucination scores using Eq. (1)
  Learn decision threshold τ on $\mathcal{D}_{\mathrm{train}}$ via Youden's J statistic
  Evaluate response-level AUC on $\mathcal{D}_{\mathrm{test}}$

Select (N^*, K^*) with the highest response-level AUC

Final evaluation:
Retrain Logistic Regression using (N^*, K^*) on $\mathcal{D}_{\mathrm{train}}$
Apply learned (α, β, τ) to $\mathcal{D}_{\mathrm{test}}$
Obtain chunk-level predictions and aggregate to response-level results

4.3.3 非負最小平方法（Non-Negative Least Squares, NNLS）

除了 Logistic Regression 之外，我們亦採用非負最小平方法（NNLS）作爲另一種線性模型，與前述方法不同，NNLS 不進行特徵彙整，也就是計算 sum，而是保留每一個 ECS head 與每一個 PKS layer，並爲每一個特徵學習獨立且非負的權重。

對於每一個 response chunk r_j，其幻覺分數定義爲：

$$\mathrm{Score}(r_j) = \sum_{i \in \mathcal{L}} \alpha_i \cdot \mathrm{PKS}_i(r_j) \; - \sum_{(\ell,h) \in \mathcal{H}} \beta_h \cdot \mathrm{ECS}_{\ell,h}(r_j), \tag{2}$$

其中 $\alpha_i \geq 0$ 與 $\beta_h \geq 0$ 分別爲 PKS layers 與 ECS heads 所對應的非負權重，由 NNLS 在訓練集上學習得到，$\mathcal{L}$ 與 $\mathcal{H}$ 表示選定的 PKS layers 與 ECS heads 集合。

在進行 NNLS 前，我們同樣僅使用訓練集對所有特徵進行 Min–Max 正規化，並將相同轉換套用至測試集。決策閾值的學習方式與 Logistic Regression 完全一致，同樣於訓練集上使用 Youden’s J statistic，並於測試階段固定使用該閾值進行預測。

完整流程如 Algorithm 7 所示。

Algorithm 7: NNLS-based Hallucination Scoring with Feature Selection

Input: Training set $\mathcal{D}_{\text{train}}$, Test set $\mathcal{D}_{\text{test}}$;
Candidate ECS head set $\mathcal{H}_{\text{cand}}$;
Candidate PKS layer set $\mathcal{L}_{\text{cand}}$
Output: Hallucination scores and predicted labels at chunk-level and response-level
Feature ranking (train only):
foreach $(\ell, h) \in \mathcal{H}_{\text{cand}}$ **do**
  Compute AUC of $\mathrm{ECS}_{\ell,h}$ on $\mathcal{D}_{\text{train}}$
foreach $i \in \mathcal{L}_{\text{cand}}$ **do**
  Compute AUC of PKS_i on $\mathcal{D}_{\text{train}}$
Sort ECS heads and PKS layers by descending AUC to obtain $\mathcal{H}_{\text{rank}}$ and $\mathcal{L}_{\text{rank}}$
Grid search over feature subsets:
foreach (N, K) **do**
  Select Top-N ECS heads $\mathcal{H} \subset \mathcal{H}_{\text{rank}}$
  Select Top-K PKS layers $\mathcal{L} \subset \mathcal{L}_{\text{rank}}$
  Apply Min–Max normalization to all selected PKS and ECS features using $\mathcal{D}_{\text{train}}$
  Train NNLS on $\mathcal{D}_{\text{train}}$ to learn non-negative weights $\{\alpha_i\}_{i \in \mathcal{L}}$ and $\{\beta_h\}_{h \in \mathcal{H}}$
  foreach $r_j \in \mathcal{D}_{\text{train}}$ **do**
    Compute hallucination score using Eq. (2)
  Learn decision threshold τ on $\mathcal{D}_{\text{train}}$ via Youden’s J statistic
  Evaluate response-level AUC on $\mathcal{D}_{\text{test}}$
Select (N^*, K^*) with the highest response-level AUC
Final evaluation:
Retrain NNLS using (N^*, K^*) on $\mathcal{D}_{\text{train}}$
Apply learned weights and threshold $(\alpha_i, \beta_h, \tau)$ to $\mathcal{D}_{\text{test}}$
Compute chunk-level scores and predictions
Aggregate chunk-level results to obtain response-level outputs

4.3.4 評估指標

我們同時於 chunk-level 與 response-level 評估模型效能。在 chunk-level 中，每一個 response chunk 視爲一筆樣本；在 response-level 中，我們將同一個 response 中所有 chunks 的幻覺分數取平均，幻覺標籤與預測結果則取最大值，以反映只要 response 中任一 chunk 出現幻覺，即視爲該 response 發生幻覺。

對於兩個層級，我們皆回報以下評估指標：AUC、Accuracy、F1-score，以及幻覺分數與標籤之間的 Pearson correlation coefficient（PCC）。

Table 1: Prompt 長度統計（字元數）

Dataset	Average Length	Maximum Length
HalluRAG	1800–1920	4100–4460
RAGTruth	1660–1690	2700–3100

Table 2: Response 長度統計（字元數）

Dataset	Average Length	Maximum Length
HalluRAG	230	1500
RAGTruth	650–690	1900–2300

5 Experimental Results

5.1 實驗設定（Experimental Setup）

5.1.1 實驗環境（Experimental Environment）

所有實驗皆於 Kaggle 平台上進行，使用單張 NVIDIA Tesla P100 GPU（16GB VRAM）。模型推論與特徵提取流程皆在相同硬體環境下完成，以排除不同運算資源所造成的效能差異。

5.1.2 資料集切分與訓練設定

在所有實驗中，我們皆遵循原始資料集所提供的資料切分設定，並將資料劃分爲訓練集與測試集。其中，訓練集與測試集的比例約爲 80% 與 20%，且所有模型訓練、特徵選擇與超參數搜尋（包含 Top-N / Top-K 特徵選擇、Min–Max 正規化參數估計、以及決策閾值的學習）皆僅於訓練集上進行。

測試集在整個實驗流程中僅用於最終模型評估，未參與任何模型選擇或參數調整，以確保評估結果的公平性並避免資訊洩漏。

5.1.3 Chunk-level 前處理設定

爲在計算效率與語意完整性之間取得平衡，我們於實驗前針對 HalluRAG 與 RAGTruth 兩個資料集的 Prompt 與 Response 進行長度統計分析，作爲 chunk 設計的依據。

Prompt 長度統計 表 1 顯示，兩個資料集的 Prompt 平均長度皆約落在 1,600 至 1,900 字元之間。其中 HalluRAG 的最大 Prompt 長度可達約 4,400 字元，顯示若不進行切分，將對模型推論與後續特徵計算造成顯著負擔。

Response 長度統計 表 2 顯示，HalluRAG 的 Response 平均長度約爲 230 字元，而 RAGTruth 則明顯較長，平均約落在 650 至 690 字元之間，最大可超過 2,000 字元。若直接以 HalluRAG 的平均長度作爲 chunk 上限，將導致 RAGTruth 被切分爲過多細碎 chunks，顯著增加後續 ECS 與 PKS 的計算成本。

Chunk 參數設定 綜合上述統計結果，以及避免將 Prompt 切分爲超過 10 個 chunks 的考量，我們最終將 Prompt 的最大 chunk 長度設定爲 **500 字元**。在 Response 方面，爲兼顧不同資料集的長度分佈並控制計算成本，我們採用 **350 字元**作爲 Response 的最大 chunk 長度。Chunk overlap 的設計則用以保留前後文語意連續性，避免關鍵資訊因切分而遺失。

5.1.4 Copy Head 選擇策略

在 Copy Head 搜尋中，我們以四個指標（PrevCopy、SelfCopy、Peak、Entropy）去計算：

$$\text{Copy score} = \alpha \cdot \text{PrevCopy} + \beta \cdot \text{SelfCopy} + \gamma \cdot \text{Peak} + \delta \cdot (1 - \text{Entropy}).$$

由於完整的 Copy Head 掃描與權重選擇在計算上成本極高，且不同模型規模的 attention 行爲具有一定一致性，因爲我們已經在 7B 模型上擁有確切的 Copy Head ，因此我們以此當作基準，去建立一組表現最佳的權重組合 $(\alpha, \beta, \gamma, \delta)$，接著將該組權重應用於 1B 與 3B 模型上，以相同的 Copy score 計算方式挑選 Copy Heads。此策略可降低重複調參的成本，並使不同模型的 Copy Head 定義與後續 ECS 特徵具有可比較性。

Table 3: ReDeEP 在 RAGTruth 資料集上的實驗結果

Method	Level	LLaMA 3.2 3B				LLaMA 3.2 1B			
		AUC	PCC	Acc.	F1	AUC	PCC	Acc.	F1
Logistic	Chunk	**0.7610**	0.3113	**0.6488**	0.3263	**0.7120**	0.2515	**0.5661**	0.2981
	Response	0.7437	**0.3238**	0.5578	**0.4024**	0.7046	**0.2554**	0.4911	**0.3893**
NNLS	Chunk	**0.7210**	**0.2660**	**0.5976**	0.3048	**0.7155**	**0.2614**	**0.5205**	0.2894
	Response	0.6868	0.2592	0.4733	**0.3646**	0.6699	0.2241	0.3956	**0.3462**

Table 4: ReDeEP 在 HalluRAG 資料集上的實驗結果

Method	Level	LLaMA 3.2 3B				LLaMA 3.2 1B			
		AUC	PCC	Acc.	F1	AUC	PCC	Acc.	F1
Logistic	Chunk	**0.8134**	**0.3494**	0.7325	0.6557	0.5760	0.1424	0.4140	0.4588
	Response	0.8124	0.3459	**0.7360**	**0.6916**	**0.6447**	**0.2379**	**0.4640**	**0.5248**
NNLS	Chunk	0.8095	0.3563	**0.7261**	0.6126	0.5937	0.1457	**0.6178**	0.4118
	Response	**0.8215**	**0.3625**	0.7200	**0.6392**	**0.5965**	**0.1563**	0.6160	**0.4667**

5.2 實驗結果（Quantitative Results）

我們在 RAGTruth 與 HalluRAG 兩個資料集上，分別以 LLaMA 3.2 1B 與 LLaMA 3.2 3B 作爲基礎模型，評估 ReDeEP 方法在 Logistic Regression 與非負最小平方法（NNLS）兩種線性分類器下的表現。所有實驗皆同時回報 *chunk-level* 與 *response-level* 的評估結果。

5.2.1 RAGTruth 上的實驗結果

表 3 彙整了 ReDeEP 在 RAGTruth 資料集上的量化結果。我們同時回報 chunk-level 與 response-level 的評估指標，並比較兩種線性分類器在不同模型規模下的表現。

5.2.2 HalluRAG 上的實驗結果（Knowledge Cut-off）

表 4 顯示 ReDeEP 在 HalluRAG 資料集上的實驗結果。此資料集刻意測試模型在 知識截止點（*knowledge cut-off*）情境下的幻覺偵測能力，因此能直接反映方法對未見過知識的泛化性。

5.3 結果分析與討論

根據上述實驗結果，我們整理出以下幾點重要觀察。

首先，在大多數實驗設定下，Logistic Regression 的整體表現優於 NNLS，特別是在 AUC 與 F1-score 等指標上。我們推測其主要原因在於，NNLS 對模型係數施加非負限制，雖有助於提升可解釋性與穩定性，但同時也降低了模型對 ECS 與 PKS 特徵中雜訊的容忍度。相較之下，Logistic Regression 不對係數符號加以限制，能更彈性地調整不同特徵的相對影響，因此在整體偵測效能上表現較佳。

其次，LLaMA 3.2 1B 模型在處理訓練資料之外的知識時，其幻覺偵測能力明顯受限。此現象在 HalluRAG 資料集上尤爲顯著，其中 chunk-level AUC 已降至 0.5760，接近隨機猜測的水準。該結果顯示，當模型容量不足時，由內部狀態提取的 ECS 與 PKS 訊號較難穩定反映幻覺行爲。

第三，ReDeEP 在較大模型上展現出一致且明顯的優勢。無論是在 RAGTruth 或 HalluRAG 上，LLaMA 3.2 3B 在所有評估指標中皆穩定優於 1B 模型。我們認爲，較大的模型具備更豐富的內部表示能力，能降低內部參數化知識訊號中的雜訊比例，使 ReDeEP 所利用的特徵更具判別力。

最後，本研究最初關注的問題在於 ReDeEP 是否能夠泛化至模型面對未見過知識的情境。從 LLaMA 3.2 3B 在 HalluRAG 上的實驗結果可見，即使在知識截止點（knowledge cut-off）的極端設定下，ReDeEP 仍能在 chunk-level 上達到超過 0.8 的 AUC，並在 response-level 維持穩定表現。此結果驗證了 ReDeEP 方法在未知知識與時間泛化場景下的實用性與泛化能力。

5.3.1 實作限制與困難（Practical Limitations）

本研究在實作層面上仍存在許多限制。首先，受限於硬體資源，所有實驗皆於單張 NVIDIA Tesla P100（16GB VRAM）上完成，因此我們選擇 LLaMA 3.2 1B 與 3B 作爲基礎模型，而非原始 ReDeEP 論文中使用的 LLaMA 2 7B 與 13B。此設定雖可在有限資源下完整重現 ReDeEP 的方法流程，但較小的模型容量可能限制了模型內部特徵（特別是 PKS）的穩定性與判別能力。

其次，在資料集選擇方面，我們僅評估 RAGTruth 中的 QA 類別資料。一方面，此設定可確保與 HalluRAG （僅包含 QA 類型）之間的比較公平性；另一方面，RAGTruth 的 summary 類別在實際進行 chunk-level 特徵提取時會消耗大量記憶體與計算資源，在現有硬體條件下難以進行完整實驗，因此被排除於本研究之外。

此外，ReDeEP 在實務應用上的一項關鍵限制來自其計算成本。ECS 與 PKS 的計算皆需存取模型內部狀態，並進行 attention 分析、語意嵌入計算，以及跨層 logits 分布比較，此類多階段計算將顯著增加推論延遲，使其在即時（online）幻覺偵測情境中仍面臨實際部署上的挑戰。

5.3.2 未來研究方向（Future Work）

針對上述限制，未來研究可從多個方向進一步提升 ReDeEP 的實用性與可擴展性。首先，可考慮降低 ECS 與 PKS 的計算成本，例如透過 attention head 或 layer 的進一步剪枝，僅保留對幻覺判別最具貢獻的內部訊號，以減少不必要的模型內部狀態存取。

最後，隨著更大模型與更高效推論硬體的普及，如果我們可以擁有的話希望可以在未來亦可重新評估 ReDeEP 於 7B 甚至更大規模模型上的表現，並進一步驗證其在知識截止點（knowledge cut-off）與眞實應用場景中的泛化能力，同時也想知道是否有 cross-dataset training（train on RAGTruth, test on HalluRAG）的可能。

6 Expected Contributions of Each Team Member

- **314511037 張周芳:** 負責程式撰寫、書面文件撰寫，以及簡報製作 (Experiment + Conclusion)，在口頭報告主要講述實驗前兩階段。
- **314511062 賴子永:** 負責程式撰寫、書面文件撰寫，以及簡報製作 (Experiment + Conclusion)，在口頭報告主要講述實驗最後一階段與結論。
- **314511054 林致葆:** 主要負責簡報製作 (Introduction + Method) 以及最終排版，書面報告檢閱，在口頭報告主要講述介紹。
- **414511002 歐陽元貞:** 負責口頭報告 Method 內容說明，以及書面報告檢閱。

References

Potsawee Manakul, Adian Liusie, and Mark JF Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. *arXiv preprint arXiv:2303.08896*, 2023.

Xiangkun Hu, Dongyu Ru, Lin Qiu, Qipeng Guo, Tianhang Zhang, Yang Xu, Yun Luo, Pengfei Liu, Yue Zhang, and Zheng Zhang. Refchecker: Reference-based fine-grained hallucination checker and benchmark for large language models. *arXiv preprint arXiv:2405.14486*, 2024.

Zhongxiang Sun, Xiaoxue Zang, Kai Zheng, Yang Song, Jun Xu, Xiao Zhang, Weijie Yu, and Han Li. Redeep: Detecting hallucination in retrieval-augmented generation via mechanistic interpretability. *arXiv preprint arXiv:2410.11414*, 2024.

Fabian Ridder and Malte Schilling. The hallurag dataset: Detecting closed-domain hallucinations in rag applications using an llm's internal states. *arXiv preprint arXiv:2412.17056*, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.

Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntong Song, and Tong Zhang. Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10862–10878, 2024.